T527 Andriod13 HDMI 转 CSI 驱动开发指南

版本号: 1.2

发布日期: 2025.09.01

广州星翼电子科技有限公司zdyzlinux

版本历史

版本号	日期	制/修订人	内容描述
1.0	2023.08.07	AWA1456	建立初版
1.1	2023.11.09	AWA1456	修订发布版本
1.2	2025.09.01	AWA2251	修改部分表达

广州星翼电子科技有限公司zdyzlinux

版权所有 © 珠海全志科技股份有限公司。保留一切权利

目　录

1 前言 **1**

1.1 文档简介 1

1.2 目标读者 1

1.3 适用范围 1

1.4 专有名词 1

2 驱动开发 **2**

2.1 新增接口说明 2

2.1.1 新增结构体 v4l2_dv_timings_cap 2

2.1.2 s_dv_timings 接口 2

2.1.3 g_dv_timings 接口 3

2.1.4 query_dv_timings 接口 3

2.1.5 enum_dv_timings 接口 4

2.1.6 dv_timings_cap 接口 4

2.2 热插拔与分辨率切换检测 5

2.2.1 驱动注册事件回调函数 5

2.2.2 dts 添加节点 7

2.2.3 驱动获取 sensor_detect 节点信息 7

2.2.4 检测数据转出，上报事件 9

2.2.5 上层注册事件，检测事件上报 10

版权所有 © 珠海全志科技股份有限公司。保留一切权利

1 前言

1.1 文档简介

介绍转换 IC 适配 VIN 驱动，即 DPIN 信号转 CSI 开发方法，其中 DPIN 信号接口包括 USB(tpye-c) 和 HDMI，当然不止这两种，也可以适用于其他输入信号接口，CSI 接口包括 MIPI-CSI 接口和 DVP 普通并口，主要介绍新增接口说明、动态分辨率切换以及热插拔开发说明，方便开发人员使用。

1.2 目标读者

camera 模块的驱动开发/维护人员。

1.3 适用范围

表 1-1: 适用产品列表

系统版本	| 驱动文件目录
Linux-5.15 |	bsp/drivers/vin

1.4 专有名词

- CSI(Camera Serial Interface)：图像串行接口，一般指串行数字接口
- MIPI(Mobile Industry Processor Interface)：高速接收接口
- VINC(Video Input Control)：视频输出 dma 接收
- USB(type-c): 一种 USB 接口外形标准
- HDMI(High Definition Multimedia Interface): 高清多媒体接口

版权所有 © 珠海全志科技股份有限公司。保留一切权利
文档问题反馈: aw-document@allwinnertech.com

2 驱动开发

本文档主要对新增接口说明、动态分辨率切换以及热插拔开发说明，不会针对转换 IC 如何驱动进行说明，因为不同厂商不同 IC 的驱动方式都是不一样的。接下里我们以龙讯转换 IC-lt7911d 为例讲解，代码位于 bsp/drivers/vin/modules/sensor/lt7911d_mipi.c。

2.1 新增接口说明

新增 V4L2 标准接口 – DV timings ioctls 接口，五个接口如下:

2.1.1 新增结构体 v4l2_dv_timings_cap

```
static const struct v4l2_dv_timings_cap lt7911d_timings_cap = {
    .type = V4L2_DV_BT_656_1120,
    .reserved = { 0 },
    V4L2_INIT_BT_TIMINGS(
            640, 3840,       /* min/max width */
            480, 2160,       /* min/max height */
            0, 400000000,    /* min/max pixelclock*/
            V4L2_DV_BT_STD_CEA861 | V4L2_DV_BT_STD_DMT |
            V4L2_DV_BT_STD_GTF | V4L2_DV_BT_STD_CVT,  /* Supported standards */
            V4L2_DV_BT_CAP_PROGRESSIVE | V4L2_DV_BT_CAP_INTERLACED |
            V4L2_DV_BT_CAP_REDUCED_BLANKING |
            V4L2_DV_BT_CAP_CUSTOM)    /* capabilities */
};
```

这里主要关注最小最大宽和高，最小最大像素时钟频率，因为将会在获取转换 IC 的 dv timings 后，与之进行对比，判断转换 IC training 成功与否，因为转换 IC training 成功就会输出信号。

2.1.2 s_dv_timings 接口

设置 dv timings，但是由于 lt7911d 是根据接入的信号进行数据转出，所以无法设置 dv timings，但是还是保留了该接口，如下:

```
static int sensor_s_dv_timings(struct v4l2_subdev *sd,
                struct v4l2_dv_timings *timings)
{
    struct sensor_info *info = to_state(sd);

    if (!timings)
        return -EINVAL;
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
    /* v4l2_print_dv_timings(sd->name, "s_dv_timings: ", timings, false); */

    if (v4l2_match_dv_timings(&info->timings, timings, 0, false)) {
        sensor_err("%s: timings no change\n", __func__);
        return 0;
    }

    if (!v4l2_valid_dv_timings(timings,
                &lt7911d_timings_cap, NULL, NULL)) {
        sensor_err("%s: timings out of range\n", __func__);
        return -ERANGE;
    }

    sensor_print("%s timings is get form input dp size, cannot set by users\n", sd->name);

    return 0;
}
```

可以操作/dev/videox 节点，并使用 VIDIOC_S_DV_TIMINGS 接口设置到转换 IC 驱动。

2.1.3 g_dv_timings 接口

获取转换 IC 的 dv timings，使用 TWI 接口读取转换 IC 的 dv timings 即可，如下：

```
static int sensor_g_dv_timings(struct v4l2_subdev *sd,
                struct v4l2_dv_timings *timings)
{
    struct sensor_info *info = to_state(sd);
    struct sensor_indetect *sensor_indet = &info->sensor_indet;

    if (!timings)
        return -EINVAL;

    mutex_lock(&sensor_indet->detect_lock);
    lt7911d_get_detected_timings(sd, timings);
    memcpy(&info->timings, timings, sizeof(*timings));
    mutex_unlock(&sensor_indet->detect_lock);

    /* v4l2_print_dv_timings(sd->name, "g_dv_timings: ", timings, false); */

    return 0;
}
```

可以操作/dev/videox 节点，并使用 VIDIOC_G_DV_TIMINGS 接口设置到转换 IC 驱动。

2.1.4 query_dv_timings 接口

获取转换 IC 的 dv timings，并判断转换 IC 是否 training 成功，如下：

```
static int sensor_query_dv_timings(struct v4l2_subdev *sd,
                struct v4l2_dv_timings *timings)
{
    struct sensor_info *info = to_state(sd);
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
    struct sensor_indetect *sensor_indet = &info->sensor_indet;

    if (!timings)
        return -EINVAL;

    mutex_lock(&sensor_indet->detect_lock);
    lt7911d_get_detected_timings(sd, timings);
    memcpy(&info->timings, timings, sizeof(*timings));

    /* v4l2_print_dv_timings(sd->name, "query_dv_timings: ", timings, false); */

    if (!v4l2_valid_dv_timings(timings, &lt7911d_timings_cap, NULL,
                NULL)) {
        sensor_err("%s: timings out of range\n", __func__);
        return -ERANGE;
    }
    mutex_unlock(&sensor_indet->detect_lock);

    return 0;
}
```

可以操作/dev/videox 节点，并使用 VIDIOC_QUERY_DV_TIMINGS 接口设置到转换 IC 驱动。

注意：
该接口是关键接口，打开/dev/videox 节点获取 dp in 数据之前需要调用该接口查看转换 IC 是否 training 成功，training 成功后可以获取到分辨率和帧率，再使用获取到的分辨率和帧率设置到 VIN。

2.1.5 enum_dv_timings 接口

枚举当前合适的 dv timings，即枚举所有 v4l2_dv_timings_presets 与 v4l2_dv_timings_cap 进行对比，如下所示：

```
static int sensor_enum_dv_timings(struct v4l2_subdev *sd,
                struct v4l2_enum_dv_timings *timings)
{
    if (timings->pad != 0)
        return -EINVAL;

    return v4l2_enum_dv_timings_cap(timings,
            &lt7911d_timings_cap, NULL, NULL);
}
```

可以操作/dev/videox 节点，并使用 VIDIOC_ENUM_DV_TIMINGS 接口设置到转换 IC 驱动。

2.1.6 dv_timings_cap 接口

获取驱动设置的 v4l2_dv_timings_cap 结构体，如下：

```
static int sensor_dv_timings_cap(struct v4l2_subdev *sd,
                struct v4l2_dv_timings_cap *cap)
{
    if (cap->pad != 0)
        return -EINVAL;
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
    *cap = lt7911d_timings_cap;

    return 0;
}
```

可以操作/dev/videox 节点，并使用 VIDIOC_DV_TIMINGS_CAP 接口设置到转换 IC 驱动。

2.2 热插拔与分辨率切换检测

热插拔与分辨率切换检测采用的是 v4l2 事件上报机制：

1. 热插拔采用的是 V4L2_EVENT_CTRL 事件，即 event_type 是 V4L2_EVENT_CTRL，event_id 是 V4L2_CID_DV_TX_HOTPLUG;
2. 分辨率切换检测采用的是 V4L2_EVENT_SOURCE_CHANGE 事件，其中:

- event_type：V4L2_EVENT_SOURCE_CHANGE；
- event_id：0;
- event.u.src_change.changes :V4L2_EVENT_SRC_CH_RESOLUTION;

2.2.1 驱动注册事件回调函数

1. sensor 注册的时候调用 probe 函数需要赋予 sensor 的 sd 为 V4L2_SUBDEV_FL_HAS_EVENTS 属性，表示支持事件上报。

```
static int sensor_probe(struct i2c_client *client,
            const struct i2c_device_id *id)
{
    struct v4l2_subdev *sd;
    struct sensor_info *info;
    info = kzalloc(sizeof(struct sensor_info), GFP_KERNEL);
    if (info == NULL)
        return -ENOMEM;
    sd = &info->sd;
    cci_dev_probe_helper(sd, client, &sensor_ops, &cci_drv);
    sd->flags |= V4L2_SUBDEV_FL_HAS_EVENTS;
    sensor_init_controls(sd, &sensor_ctrl_ops);
    mutex_init(&info->lock);
    ......
    sensor_det_init(sd);
    ......
    return 0;
}
```

2. 驱动添加事件注册回调函数，支持上层注册事件到底层。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
static int sensor_subscribe_event(struct v4l2_subdev *sd, struct v4l2_fh *fh,
                    struct v4l2_event_subscription *sub)
{
    switch (sub->type) {
    case V4L2_EVENT_SOURCE_CHANGE:
        sensor_dbg("sensor subscribe event -- V4L2_EVENT_SOURCE_CHANGE\n");
        return v4l2_src_change_event_subdev_subscribe(sd, fh, sub);
    case V4L2_EVENT_CTRL:
        sensor_dbg("sensor subscribe event -- V4L2_EVENT_CTRL\n");
        return v4l2_ctrl_subdev_subscribe_event(sd, fh, sub);
    default:
        return -EINVAL;
    }
}
```

3. 由于热插拔信号采用的是 V4L2_EVENT_CTRL 事件，所以还需要创建 V4L2_CID_DV_TX_HOTPLUG controls，在 sensor_init_controls 函数中创建，如下：

```
static int sensor_init_controls(struct v4l2_subdev *sd, const struct v4l2_ctrl_ops *ops)
{
    struct sensor_info *info = to_state(sd);
    struct v4l2_ctrl_handler *handler = &info->handler;
    struct v4l2_ctrl *ctrl;
    int ret = 0;

    v4l2_ctrl_handler_init(handler, 2+1);

    ctrl = v4l2_ctrl_new_std(handler, ops, V4L2_CID_GAIN, 1 * 1600,
                256 * 1600, 1, 1 * 1600);
    if (ctrl != NULL)
        ctrl->flags |= V4L2_CTRL_FLAG_VOLATILE;

    ctrl = v4l2_ctrl_new_std(handler, ops, V4L2_CID_EXPOSURE, 0,
                65536 * 16, 1, 0);
    if (ctrl != NULL)
        ctrl->flags |= V4L2_CTRL_FLAG_VOLATILE;

    info->sensor_indet.ctrl_hotplug = v4l2_ctrl_new_std(handler, ops, V4L2_CID_DV_TX_HOTPLUG,
                HOTPLUT_DP_OUT, HOTPLUT_DP_MAX, 0, HOTPLUT_DP_OUT);

    if (handler->error) {
        ret = handler->error;
        v4l2_ctrl_handler_free(handler);
    }

    sd->ctrl_handler = handler;

    return ret;

}
```

其中：

(1) v4l2_ctrl_handler_init 函数表示需要创建 controls 的数量，需要根据实际填写即可；

(2) V4L2_CID_DV_TX_HOTPLUG 参数使用的是 vin 自己定义的，如下：

```
enum hotplut_state {
    HOTPLUT_DP_OUT = 0,
    HOTPLUT_DP_IN,
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
    HOTPLUT_DP_NOSUPPRT,
    HOTPLUT_DP_MAX = 0x3,
};
```

- HOTPLUT_DP_OUT：表示热插拔信号拔出
- HOTPLUT_DP_IN：表示热插拔信号插入
- HOTPLUT_DP_NOSUPPRT：表示热插拔型号插入，但是驱动目前不支持，需要在驱动 win_size 添加多一组该配置；
- HOTPLUT_DP_MAX：为所有情况的与的结果；

4. v4l2_ctrl_ops 添加 try_ctrl

```
static const struct v4l2_ctrl_ops sensor_ctrl_ops = {
    ......
    .try_ctrl = sensor_try_ctrl,
};
```

添加 try_ctrl 的目的是为了上报 V4L2_EVENT_CTRL–V4L2_CID_DV_TX_HOTPLUG 事件，使其可以在间隔两次事件中传输同一个 enum hotplut_state 参数；

2.2.2 dts 添加节点

由于需要在 sensor 注册后马上检测热插拔信号和分辨率动态切换，所以需要添加新的节点获取引脚信息以便可以在 sensor_probe 函数可以获取引脚信息。

- hotplug_gpios 引脚：热插拔检测引脚，主控与转换 IC 相连接引脚，在数据转出或者分辨率变化的时候转换 IC 会输出高电平信号（或者低电平信号，因不同转换 IC 而有差异），所以可以注册与主控相连接的引脚的中断处理函数，检测引脚高电平信号（或者低电平信号），从而检测热插拔或者分辨率动态切换。
- power_gpios 引脚：转换 IC 上电引脚，只有转换 IC 上电后才可以检测 hotplug_gpios 引脚。

```
&vind0 {
    ......
    sensor_detect: sensor_detect@2108200 {
        hotplug_gpios = <&pio PI 12 GPIO_ACTIVE_LOW>;
        power_gpios = <&pio PI 11 GPIO_ACTIVE_LOW>;
    };
    ......
};
```

2.2.3 驱动获取 sensor_detect 节点信息

1. 获取 power_gpios 引脚并拉高给转换 IC 上电；
2. 获取 hotplug_gpios 引脚，设置为 input 模式，注册中断函数，检测上拉电平信号。这里还需要注意一点，中断回调函数被调用后，证明有数据转出或者分辨率变化，这时候需要使用 twi 读取转换 IC 的信息，所以只能使用工作队列的方式来使用 twi 读取转换 IC 的信息（中断回调函数无法内嵌中断）。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
static int sensor_det_init(struct v4l2_subdev *sd)
{
    int ret;
    struct device_node *np = NULL;
    enum of_gpio_flags gc;
    struct sensor_info *info = to_state(sd);
    struct sensor_indetect *sensor_indet = &info->sensor_indet;
    char *node_name = "sensor_detect";
    char *hotplug_gpio_name = "hotplug_gpios";
    char *power_gpio_name = "power_gpios";

    np = of_find_node_by_name(NULL, node_name);
    if (np == NULL) {
        sensor_err("can not find the %s node\n", node_name);
        return -EINVAL;
    }

    /* power on sensor to detect hotplug*/
    sensor_indet->power_gpio.gpio = of_get_named_gpio_flags(np, power_gpio_name, 0, &gc);
    sensor_dbg("get form %s gpio is %d\n", power_gpio_name, sensor_indet->power_gpio.gpio);
    if (!gpio_is_valid(sensor_indet->power_gpio.gpio)) {
        sensor_err("fetch %s from device_tree failed\n", power_gpio_name);
        return -ENODEV;
    } else {
        ret = gpio_request(sensor_indet->power_gpio.gpio, NULL);
        if (ret < 0) {
            sensor_err("request %s fail!\n", power_gpio_name);
            return -1;
        }
        gpio_direction_output(sensor_indet->power_gpio.gpio, 1);
    }
    sd->entity.use_count++;

    sensor_indet->hotplug_det_gpio.gpio = of_get_named_gpio_flags(np, hotplug_gpio_name, 0, &
    gc);
    sensor_dbg("get form %s gpio is %d\n", hotplug_gpio_name, sensor_indet->hotplug_det_gpio.
    gpio);
    if (!gpio_is_valid(sensor_indet->hotplug_det_gpio.gpio)) {
        sensor_err("fetch %s from device_tree failed\n", hotplug_gpio_name);
        return -ENODEV;
    } else {
        ret = gpio_request(sensor_indet->hotplug_det_gpio.gpio, NULL);
        if (ret < 0) {
            sensor_err("request %s fail!\n", hotplug_gpio_name);
            return -1;
        }
        gpio_direction_input(sensor_indet->hotplug_det_gpio.gpio);

        sensor_indet->hotplug_det_irq = gpio_to_irq(sensor_indet->hotplug_det_gpio.gpio);
        if (sensor_indet->hotplug_det_irq <= 0) {
            sensor_err("gpio %d get irq err\n", sensor_indet->hotplug_det_gpio.gpio);
            return -1;
        }
        ret = request_irq(sensor_indet->hotplug_det_irq, sensor_det_irq_func,
                IRQF_TRIGGER_RISING,/* IRQF_TRIGGER_FALLING | IRQF_TRIGGER_RISING, */
                "sensor_detect", sd);
    }

    INIT_WORK(&sensor_indet->sensor_work, sensor_det_work);
    mutex_init(&sensor_indet->detect_lock);
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
    sensor_dbg("%s seccuss\n", __func__);
    return 0;
}
```

2.2.4 检测数据转出，上报事件

1. 热插拔信号：采用的是 V4L2_EVENT_CTRL 事件，即 event_type 是 V4L2_EVENT_CTRL，event_id 是 V4L2_CID_DV_TX_HOTPLUG，调用方式：

```
    v4l2_ctrl_s_ctrl(info->sensor_indet.ctrl_hotplug, HOTPLUT_DP_OUT);
```

2. 分辨率切换检测采用的是 V4L2_EVENT_SOURCE_CHANGE 事件，其中:

- event_type: V4L2_EVENT_SOURCE_CHANGE;
- event_id: 0;
- event.u.src_change.changes: V4L2_EVENT_SRC_CH_RESOLUTION;

```
    struct v4l2_event sensor_ev_fmt;
    sensor_ev_fmt.id = 0;
    sensor_ev_fmt.type = V4L2_EVENT_SOURCE_CHANGE,
    sensor_ev_fmt.u.src_change.changes = V4L2_EVENT_SRC_CH_RESOLUTION;
    sensor_print("%s send resolution change to user\n", sd->name);
```

3. 示例：

```
static void sensor_det_work(struct work_struct *work)
{
    struct sensor_indetect *sensor_indet = container_of(work, struct sensor_indetect,
    sensor_work);
    struct sensor_info *info = container_of(sensor_indet, struct sensor_info, sensor_indet);
    struct v4l2_subdev *sd = &info->sd;
    struct v4l2_dv_timings timings;
    struct v4l2_event sensor_ev_fmt;
    unsigned int changes = 0;
    unsigned int stream_count;

    mutex_lock(&sensor_indet->detect_lock);

    lt7911d_get_detected_timings(sd, &timings);

    if (v4l2_match_dv_timings(&info->timings, &timings, 0, false)) {
        sensor_err("%s: timings no change\n", __func__);
    } else {
        if (!v4l2_valid_dv_timings(&timings, &lt7911d_timings_cap, NULL, NULL)) {
            sensor_err("%s: timings out of range\n", __func__);
            v4l2_ctrl_s_ctrl(info->sensor_indet.ctrl_hotplug, HOTPLUT_DP_OUT); /* not signal
    -- dp out */
            sensor_print("%s send hotplug dp out to user\n", sd->name);
        } else {
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
            if (!sensor_detect_res(timings.bt.width, timings.bt.height, fps_calc(&timings.bt)
    )) {
                sensor_err("%s: driver not support %dfps@%dx%d, need add to sensor win_size\n
    ",
                            __func__, fps_calc(&timings.bt), timings.bt.width, timings.bt.
    height);

                v4l2_ctrl_s_ctrl(info->sensor_indet.ctrl_hotplug, HOTPLUT_DP_NOSUPPRT); /*
    signal -- resolution not support */
                sensor_print("%s send hotplug not support resolution to user\n", sd->name);
            } else  {
#if LINUX_VERSION_CODE >= KERNEL_VERSION(6, 1, 0)
                stream_count = info->stream_count;
#else
                stream_count = sd->entity.stream_count;
#endif
                if (stream_count) {
                    changes |= V4L2_EVENT_SRC_CH_RESOLUTION;  /* signal -- resolution change
    */
                } else {
                    v4l2_ctrl_s_ctrl(info->sensor_indet.ctrl_hotplug, HOTPLUT_DP_IN); /*
    signal -- dp in */
                    sensor_print("%s send hotplug dp in to user\n", sd->name);
                }
            }
            if (changes) {
                sensor_ev_fmt.id = 0;
                sensor_ev_fmt.type = V4L2_EVENT_SOURCE_CHANGE,
                sensor_ev_fmt.u.src_change.changes = changes;
                v4l2_subdev_notify_event(sd, &sensor_ev_fmt);
                sensor_print("%s send resolution change to user\n", sd->name);
            }
        }
    }
    memcpy(&info->timings, &timings, sizeof(timings));

    mutex_unlock(&sensor_indet->detect_lock);
}
```

2.2.5 上层注册事件，检测事件上报

1. open 转换 IC 的子设备节点

```
#define SENSOR_NAME "lt7911d_mipi"

static int subdev_open(int *sub_fd, char *str)
{
    char subdev[20] = {'\0'};
    char node[50] = {'\0'};
    char data[20] = {'\0'};
    int i, fs = -1;

    for (i = 0; i < 255; i++) {
        sprintf(node, "/sys/class/video4linux/v4l-subdev%d/name", i);
        fs = open(node, O_RDONLY/* required */| O_NONBLOCK, 0);
        if (fs < 0) {
            printf("open %s falied\n", node);
```

```
            continue;
        }
        /* data_length = lseek(fd, 0, SEEK_END); */
        lseek(fs, 0L, SEEK_SET);
        read(fs, data, 20);
        close(fs);
        if (!strncmp(str, data, strlen(str))) {
            sprintf(subdev, "/dev/v4l-subdev%d", i);
            printf("find %s is %s\n", str, subdev);
            *sub_fd = open(subdev, O_RDWR | O_NONBLOCK, 0);
            if (*sub_fd < 0) {
                printf("open %s falied\n", str);
                return -1;
            }
            printf("open %s fd = %d\n", str, *sub_fd);
            return 0;
        }
    }
    printf("can not find %s!\n", str);
    return -1;
}

static int camera_init(int sel, int mode)
{
    ......
    if (-1 == subdev_open(&sensor_fd, SENSOR_NAME))
        return -1;
    ......
}
```

2. 注册事件和反注册事件

```
static int subdev_set_subscribe_event(void)
{
    struct v4l2_event_subscription event_sub;

    CLEAR(event_sub);
    event_sub.id = 0;
    event_sub.type = V4L2_EVENT_SOURCE_CHANGE;
    if (-1 == ioctl(sensor_fd, VIDIOC_SUBSCRIBE_EVENT, &event_sub)) {
        printf("VIDIOC_SUBSCRIBE_EVENT V4L2_EVENT_SOURCE_CHANGE error!\n");
        return -1;
    }

    CLEAR(event_sub);
    event_sub.id = V4L2_CID_DV_TX_HOTPLUG;
    event_sub.type = V4L2_EVENT_CTRL;
    if (-1 == ioctl(sensor_fd, VIDIOC_SUBSCRIBE_EVENT, &event_sub)) {
        printf("VIDIOC_SUBSCRIBE_EVENT V4L2_EVENT_CTRL error!\n");
        return -1;
    }
}

static int subdev_set_unsubscribe_event(void)
{
    struct v4l2_event_subscription event_sub;

    CLEAR(event_sub);
    event_sub.id = 0;
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
    event_sub.type = V4L2_EVENT_SOURCE_CHANGE;
    if (-1 == ioctl(sensor_fd, VIDIOC_UNSUBSCRIBE_EVENT, &event_sub)) {
        printf("VIDIOC_UNSUBSCRIBE_EVENT V4L2_EVENT_SOURCE_CHANGE error!\n");
        return -1;
    }

    CLEAR(event_sub);
    event_sub.id = V4L2_CID_DV_TX_HOTPLUG;
    event_sub.type = V4L2_EVENT_CTRL;
    if (-1 == ioctl(sensor_fd, VIDIOC_UNSUBSCRIBE_EVENT, &event_sub)) {
        printf("VIDIOC_UNSUBSCRIBE_EVENT V4L2_EVENT_CTRL error!\n");
        return -1;
    }
}

static int camera_init(int sel, int mode)
{
    ......
    subdev_set_subscribe_event();
    ......
}
```

3. 检测底层的事件上报

```
static void *subdev_wait_event(void *arg)
{
    struct v4l2_event *event = arg;
    struct pollfd arg_fd;
    int ms_timeout;
    int r;

    arg_fd.fd = sensor_fd;
    arg_fd.events = POLLPRI | POLLIN;
    arg_fd.revents = 0;

    if (!event) {
        printf("error param(event) input!\n");
        return NULL;
    }

    while (!wait_exit) {
        ms_timeout = 2000;
        r = poll(&arg_fd, 1, ms_timeout);

        if (-1 == r) {
            printf("select err\n");
            return NULL;
        }
        if (0 == r) {
            //printf("subdev wait event timeout\n");
            continue;
        }

        if (-1 == ioctl(sensor_fd, VIDIOC_DQEVENT, event)) {
            printf("VIDIOC_DQEVENT error!\n");
            continue;
        }
        if (event->type == V4L2_EVENT_CTRL) {
            if (event->u.ctrl.value == HOTPLUT_DP_OUT)
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
                printf("subdev wait event until type:%d come, id is 0x%x, DP out(ctrl value
    is %d)\n", event->type, event->id, event->u.ctrl.value);
            else if (event->u.ctrl.value == HOTPLUT_DP_IN)
                printf("subdev wait event until type:%d come, id is 0x%x, DP in(ctrl value is
     %d)\n", event->type, event->id, event->u.ctrl.value);
            else
                printf("subdev wait event until type:%d come, id is 0x%x, resolution not
    support(ctrl value is %d)\n", event->type, event->id, event->u.ctrl.value);
        } else
            printf("subdev wait event until type:%d come, id is 0x%x, resolution change(
    changes is %d)\n", event->type, event->id, event->u.src_change.changes);
    }

    return 0;
}
```

广州星翼电子科技有限公司zdyzlinux

著作权声明

版权所有 ©2025 珠海全志科技股份有限公司。保留一切权利。

本文档及内容受著作权法保护，其著作权由珠海全志科技股份有限公司（“全志”）拥有并保留一切权利。

本文档是全志的原创作品和版权财产，未经全志书面许可，任何单位和个人不得擅自摘抄、复制、修改、发表或传播本文档内容的部分或全部，且不得以任何形式传播。

商标声明

ALLWINNER、ALLWINNER 全志科技、全志科技、ALLWINNER（不完全列举）均为珠海全志科技股份有限公司的商标或者注册商标。在本文档描述的产品中出现的其它商标，产品名称，和服务名称，均由其各自所有人拥有。

免责声明

您购买的产品、服务或特性应受您与珠海全志科技股份有限公司（“全志”）之间签署的商业合同和条款的约束。本文档中描述的全部或部分产品、服务或特性可能不在您所购买或使用的范围内。使用前请认真阅读合同条款和相关说明，并严格遵循本文档的使用说明。您将自行承担任何不当使用行为（包括但不限于如超压，超频，超温使用）造成的不利后果，全志概不负责。

本文档作为使用指导仅供参考。由于产品版本升级或其他原因，本文档内容有可能修改，如有变更，恕不另行通知。全志尽全力在本文档中提供准确的信息，但并不确保内容完全没有错误，因使用本文档而发生损害（包括但不限于间接的、偶然的、特殊的损失）或发生侵犯第三方权利事件，全志概不负责。本文档中的所有陈述、信息和建议并不构成任何明示或暗示的保证或承诺。

本文档未以明示或暗示或其他方式授予全志的任何专利或知识产权。在您实施方案或使用产品的过程中，可能需要获得第三方的权利许可。请您自行向第三方权利人获取相关的许可。全志不承担也不代为支付任何关于获取第三方许可的许可费或版税（专利税）。全志不对您所使用的第三方许可技术做出任何保证、赔偿或承担其他义务。